

Vision Based Object Detection and Pose Estimation*

*Team Project under Supervision of Prof. Markus Ryll

Baran Özer

Mechatronics, Robotics and Biomechanical Engineering M.Sc.

Technical University of Munich

Munich, Germany

Matriculation Nr: 03783103

baran.oezer@tum.de

Abstract—This paper presents the development of a fully autonomous drone for Funke Fully Autonomous Flight Challenge organized by SPRIN-D (Bundesagentur für Sprunginnovationen). The challenge was about designing a drone, capable of flying in an environment with obstacles, identify and follow the numbered waypoints, locate a specific box and transport it to the final destination. The focus of this work was on the detection of the waypoint numbers, detection of the flags that indicate the location of waypoints, and estimating the pose of the box for robotic grasping. For the waypoint number detection and flag detection, YOLOv8 (You Only Look Once) detection model was trained, and for the box pose estimation, YOLOv8-pose model was trained for detecting the box corners and using PNP (Perspective-n-Point) algorithm, the pose was estimated. The results demonstrated that these algorithms are both effective and real-time capable in complex and dynamic environments.

Index Terms—Computer Vision, Object Detection, Pose Estimation, Deep Learning

I. INTRODUCTION

Thanks to advancements in artificial intelligence (AI) and machine learning, autonomous systems have improved very quickly and created new solutions in variety of industries. Drones, often referred to as unmanned aerial vehicles (UAVs), are a key technology in this revolution, with a wide range of uses in fields like search and rescue, surveillance, delivery services, and environmental monitoring. Among the most difficult parts of UAV development is obtaining full autonomy, which allows drones to navigate and perform tasks in complicated situations without human intervention.

The Funke Fully Autonomous Flight Challenge, organized by SPRIN-D (Bundesagentur für Sprunginnovationen), aims for pushing the boundaries of autonomous drone technology. The challenge requires participants to develop a drone capable of flying on its own through an environment with obstacles, while detecting and following numbered waypoints, locating a specific target box, and transferring it to the final destination.

Deep learning, an advanced subset of machine learning that uses multi-layered neural networks, has become essential for computer vision applications such as object detection and pose estimation.

In this project, deep learning was employed to address the critical components of the challenge: detecting waypoint numbers, detecting flags and estimating the pose of the box for robotic grasping. The YOLOv8 (You Only Look Once) model, a state-of-the-art deep learning-based object detection algorithm, was chosen for its ability to quickly and accurately detect the numbered waypoints and flags. For the box pose estimation, the YOLOv8-pose model, an extension of YOLOv8 designed for detecting object keypoints, was used to detect the 2D image coordinates of the corners of the box. These coordinates were then utilized in the Perspective-n-Point (PNP) algorithm in order to estimate the 3D pose of the box, which is essential to be able to grasp the box.

The following sections of this paper examine the theoretical foundations of deep learning, specifically how models like YOLOv8 are designed and trained. In the last section the results show that these deep learning-based algorithms are both effective and real-time capable.

II. THEORETICAL BACKGROUND AND APPROACH

A. Deep Learning

Deep learning is a branch of machine learning that utilizes neural networks with multiple layers to model and understand complex patterns in the data. In computer vision tasks, such as object detection and pose estimation, deep learning has proven to be extremely powerful.

The artificial neural networks (ANN) are the foundation of deep learning. They consist of layers of interconnected nodes, which are also called neurons. Each neuron performs a simple computation: it receives inputs, applies a linear transformation, then a non-linear activation function, and finally produces an output.

The computation within a neuron can be mathematically expressed as:

$$z = \mathbf{w} \cdot \mathbf{x} + b \quad (1)$$

$$a = \sigma(z) \quad (2)$$

Where:

- $\mathbf{x}$ is the input vector.
- $\mathbf{w}$ is the weight vector.
- b is the bias term.
- z is the linear combination of inputs and weights.
- $\sigma(z)$ is the activation function applied to z .
- a is the output of the neuron.

One of the most common activation functions is the Rectified Linear Unit (ReLU), which makes z equal to zero if it becomes negative:

$$\sigma(z) = \text{ReLU}(z) = \max(0, z) \quad (3)$$

B. Convolutional Neural Networks (CNNs)

YOLO (You Only Look Once) is a deep learning model for object detection that uses Convolutional Neural Networks (CNNs). CNNs are a type of deep learning model that is especially appropriate to analyze visual data.

The basic procedure of a CNN is convolution, which applies a filter (or kernel) to the input data to recognize characteristics like edges, textures, or individual objects. The mathematical expression for a convolution operation is:

$$s(i, j) = (x * w)(i, j) = \sum_m \sum_n x(i + m, j + n)w(m, n) \quad (4)$$

Where:

- x is the input image.
- w is the convolutional kernel.
- $s(i, j)$ is the output feature map at position (i, j) .

C. YOLO Architecture

YOLOv8, like previous versions, leverages a fully convolutional network in order to estimate the bounding boxes and class probabilities from the images in a single evaluation. This enables YOLO to be highly rapid while maintaining the balance between accuracy and speed.

YOLO operates by dividing the input picture into a $S \times S$ grid and predicting B bounding boxes and confidence scores for each cell. Each bounding box consists of five components:

$$\mathbf{b} = (x, y, w, h, c) \quad (5)$$

Where:

- (x, y) are the coordinates of the center of the box relative to the grid cell.
- w and h are the width and height of the box relative to the whole image.
- c is the confidence score, representing the probability that the box contains an object and the accuracy of the bounding box.

In addition, YOLO predicts a class probability $p(c_i)$ for each of the C object classes.

The final prediction for a given box is computed as:

$$p(c_i) \times c \times \text{bbox} = (x, y, w, h) \quad (6)$$

The loss function in YOLO combines multiple components, including localization loss (for x, y, w, h), confidence loss (for c), and classification loss (for $p(c_i)$):

$$\begin{aligned} \text{Loss} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_{\text{conf}} \sum_{i=0}^{S^2} \sum_{j=0}^B (c_i - \hat{c}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbf{1}_i^{\text{obj}} \sum_{c=0}^C (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (7)$$

Where:

- $\mathbf{1}_{ij}^{\text{obj}}$ is an indicator function that equals 1 if object j is present in cell i , otherwise 0.
- λ_{coord} and λ_{conf} are hyperparameters that balance the different parts of the loss.

D. YOLOv8-pose

YOLOv8-pose extends the conventional YOLO framework for the pose estimation by estimating the corners of the box. The estimated 2D image coordinates of the corners are then used in the pose estimation process to determine the orientation and position of the box in 3D space using the Perspective-n-Point (PnP) method.

1) *Keypoint Detection*: The YOLOv8-pose model estimates the coordinates of specific keypoints associated with the object and these keypoints are expressed as:

$$\mathbf{k}_i = (x_i, y_i) \quad (8)$$

Where:

- $\mathbf{k}_i$ represents the i -th keypoint.
- (x_i, y_i) are the 2D coordinates of the keypoint in the image plane.

2) *Pose Estimation Using PnP*: Once the 2D image coordinates of the corners are estimated, the pose of the box can be determined by solving the Perspective-n-Point (PnP) problem, which is finding the rotation matrix $\mathbf{R}$ and translation vector $\mathbf{t}$ that map the 3D points onto the 2D image plane. The following equation represents this relationship:

$$s \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \mathbf{K} \left(\mathbf{R} \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} + \mathbf{t} \right) \quad (9)$$

Where:

- s is a scaling factor.
- $(x_i, y_i, 1)$ are the homogeneous coordinates of the 2D keypoint.
- $\mathbf{K}$ is the camera intrinsic matrix, which includes focal length and optical center parameters.
- $\mathbf{R}$ is the rotation matrix that describes the orientation of the object in 3D space.

- $\mathbf{t}$ is the translation vector that describes the position of the object in 3D space.
- (X_i, Y_i, Z_i) are the 3D coordinates of the keypoint on the object.

The goal of the PnP algorithm is to find the optimal $\mathbf{R}$ and $\mathbf{t}$ that minimize the reprojection error, which is the difference between the observed 2D keypoints (x_i, y_i) and the projected 2D points using the calculated $\mathbf{R}$ and $\mathbf{t}$.

$$\text{Reprojection Error} = \sum_{i=1}^n \left\| \mathbf{k}_i - \hat{\mathbf{k}}_i \right\|^2 \quad (10)$$

Where:

- $\mathbf{k}_i$ are the observed 2D keypoints.
- $\hat{\mathbf{k}}_i$ are the projected 2D points using the estimated $\mathbf{R}$ and $\mathbf{t}$.
- n is the number of keypoints used for pose estimation.

III. DATA GENERATION

A. Flag Detection Data Generation

For the flag detection model, 4000 labelled images were used from OpenImages database.

A part of the dataset can be seen below:

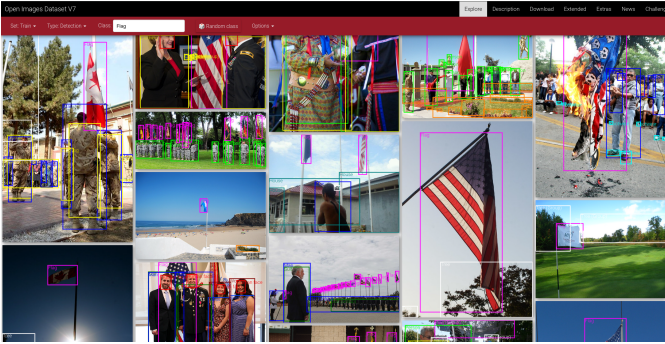


Fig. 1. OpenImages Dataset.

B. Number Detection Data Generation

For the number detection model, the public SVHN dataset was used. This is a huge amount (600,000 digit images) of labelled data of Google. They created this dataset by taking the pictures of the numbers on many people's houses.

A part of the dataset can be seen below:



Fig. 2. SVHN Dataset.

C. Box Pose Estimation Data Generation

For the box pose estimation model, 3000 box images were created synthetically. The procedure is explained below:

1) *Drawing the 3D Model of the Box in a CAD Software:* The 3D model of the box was drawn in Blenderproc, which is a software that is mainly used for photorealistic rendering. The 3D model of the box can be seen below:

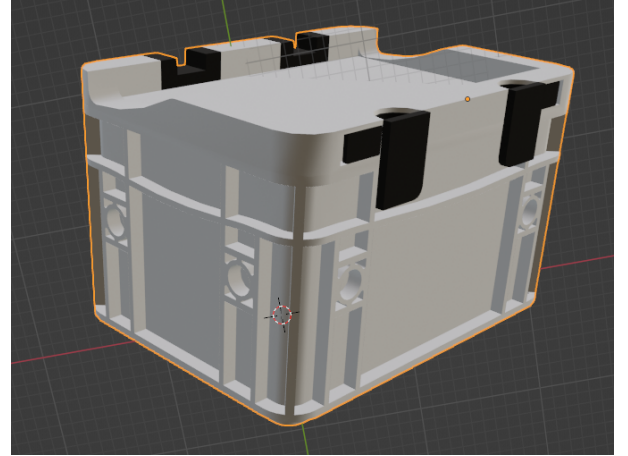


Fig. 3. Box 3D Model.

2) *Downloading Random Backgrounds for Synthetic Image Generation:* Background images were downloaded from the website "polyhaven.com/hdri" in the HDRI format.

3) *Generating the Synthetic Box Data:* The synthetic data were generated by by means of Blenderproc software, using a script. In this script, first a random background image is chosen, then the 3D box model is projected onto this background with a random orientation by using the rendering functionalities of Blenderproc.

Also, since the position of every point is known in the 3D box model, the projected coordinates of the 2D box on the image was determined, so that the keypoints on every synthetic image are labelled automatically.

IV. TRAINING AND RESULTS

A. Flag Detection Results

A YOLOv8 model was trained to detect flags, under which the waypoint locations are located. For the training data, OpenImages-v7 flag dataset was used, which is a previously labelled dataset of Google. The results show high accuracy and speed, which indicate that they are appropriate for real-time systems.

Below are the training graph and a part of the validated data for the flag detection algorithm.

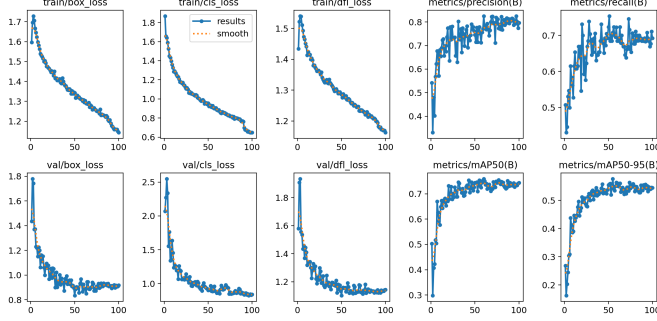


Fig. 4. Flag Detection Training Results.



Fig. 5. Flag Detection Validation.

B. Number Detection Results

A YOLOv8 model was trained to waypoint numbers which are located under the flags. For the training data, SVHN dataset was used, which is a huge amount (600,000 digit images) of labelled data of Google. The results show high accuracy and speed, which indicate that they are appropriate for real-time systems.

Below the training graph, a part of the validation data for the number detection algorithm, and the confusion matrix can be seen.

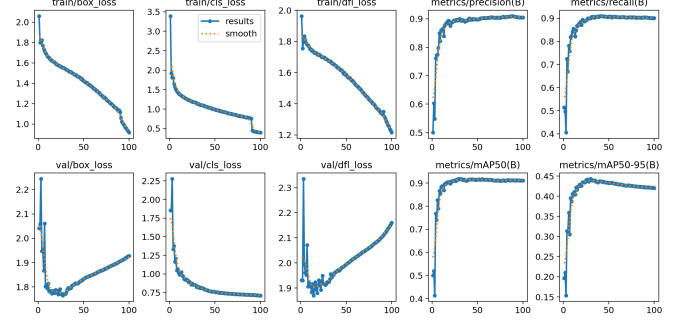


Fig. 6. Number Detection Training Results.



Fig. 7. Number Detection Validation.

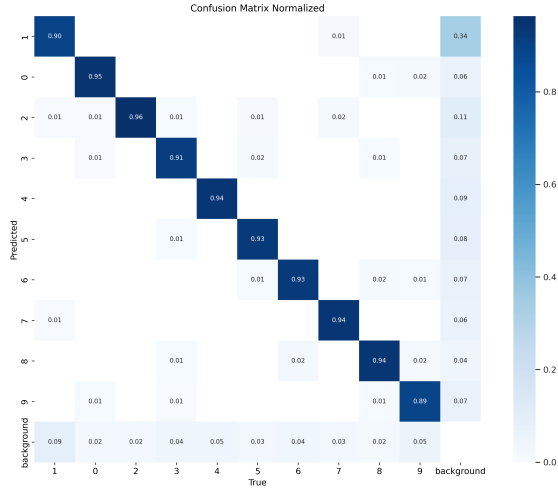


Fig. 8. Number Detection Confusion Matrix.

C. Pose Estimation Results

A YOLOv8-pose model was trained to detect the corners of the box, then using the 2D image coordinates of the corners in PNP, the box pose was determined. For the training data, 3D model of the box was drawn and 3000 synthetic images were generated with random backgrounds and random orientations.

Below the training graph, a part of the validation data, and pose estimation can be seen.

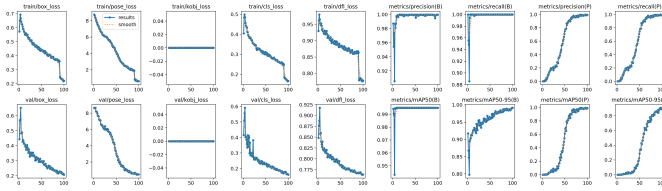


Fig. 9. Box Corner Detection Training Results.

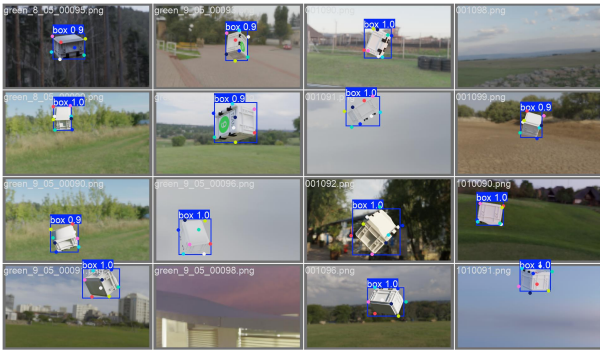


Fig. 10. Box Corner Detection Validation.

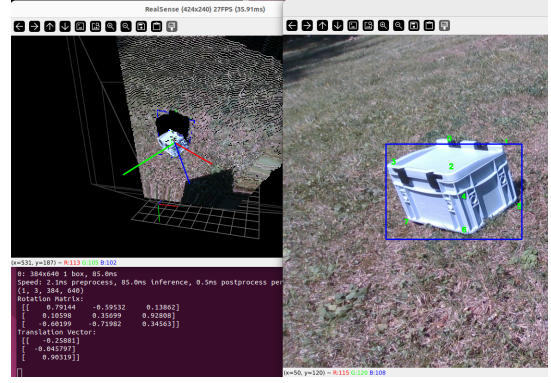


Fig. 11. Box Pose Estimation.

V. CONCLUSION

This paper presented the implementation of computer vision based object detection and pose estimation for the Funke Fully Autonomous Flight Challenge, organized by SPRIN-D. The challenge involved developing a drone that is capable of doing autonomous navigation without collision, detecting flags, waypoint numbers, and estimating the pose of the target box for robotic grasping.

The work focused on the detection and pose estimation tasks using deep learning, since it has proven success in these areas. Specifically, YOLOv8 was used for detecting waypoint numbers and flags, while YOLOv8-pose was utilized to estimate the 3D pose of the box based on detected keypoints, which are the corners of the box. The results showed that these models performed accurate detections rapidly, which makes them appropriate to real-time systems.

REFERENCES

- [1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," version 8.0.0, Ultralytics, Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>. [Accessed: Aug. 31, 2024].
- [2] Itseez, "Open Source Computer Vision Library," 2015. [Online]. Available: <https://github.com/itseez/opencv>. [Accessed: Aug. 31, 2024].
- [3] Intel Corporation, "Intel RealSense SDK 2.0," GitHub repository, 2018. [Online]. Available: <https://github.com/IntelRealSense/librealsense>. [Accessed: Aug. 31, 2024].
- [4] Wikipedia, "Perspective-n-Point," Wikipedia, The Free Encyclopedia. [Online]. Available: <https://en.wikipedia.org/wiki/Perspective-n-Point>. [Accessed: Aug. 31, 2024].
- [5] A. Netzer, T. Wang, A. Coates, B. B. Wu, and A. Y. Ng, "Reading Digits in Natural Images with Unsupervised Feature Learning," Stanford University, [Online]. Available: <https://ufldl.stanford.edu/housenumbers/>. [Accessed: Aug. 31, 2024].
- [6] Poly Haven, "HDRIs," Poly Haven, [Online]. Available: <https://polyhaven.com/hdris>. [Accessed: Aug. 31, 2024].
- [7] Open Images Dataset V7, "Open Images Dataset," Google, [Online]. Available: <https://storage.googleapis.com/openimages/web/index.html>. [Accessed: Aug. 31, 2024].